

Desenvolvimento de APIs Modernas
com Fastify

Do Setup ao Deploy

Um guia completo para construir, testar e publicar APIs profissionais utilizando Node.js e Fastify



Sumário

01

Introdução à Tecnologia Fastify

Conhecendo o framework e seus diferenciais competitivos

02

Ferramentas e Ambiente

Configuração completa do ambiente de desenvolvimento

03

Setup do Projeto

Inicialização e estruturação da aplicação

04

Estrutura da Aplicação

Organização de pastas e arquivos do projeto

05

Desenvolvimento CRUD

Criação completa da API de tarefas

06

Testes da API

Validação de endpoints e funcionalidades

07

Versionamento Git

Controle de versão e colaboração em equipe

08

Deploy na Render

Publicação da aplicação em produção

09

Conclusão

Reflexões e próximos passos do projeto

Introdução ao Fastify

O Fastify é um framework web extremamente rápido e moderno para Node.js, criado com foco em alta performance, simplicidade e produtividade. Desenvolvido por Matteo Collina, engenheiro renomado da comunidade Node.js, ele surgiu como uma alternativa mais eficiente ao Express, que, apesar de popular, apresentava limitações de desempenho em cenários de alta demanda. Sua criação visou suprir a necessidade de um framework que oferecesse melhor throughput e menor latência, especialmente para a construção de APIs e microsserviços.

Matteo Collina, um dos principais colaboradores e membros do Technical Steering Committee do Node.js, liderou o desenvolvimento do Fastify. A equipe por trás do projeto é composta por uma comunidade global de desenvolvedores que contribuem ativamente para o seu aprimoramento, garantindo que o framework se mantenha atualizado com as melhores práticas e inovações do ecossistema Node.js.

A principal característica do Fastify é sua arquitetura baseada em **plugins**. Diferente das abordagens de middleware mais tradicionais (como no Express), onde os middlewares são uma sequência de funções que processam a requisição, os plugins do Fastify oferecem um sistema de encapsulamento que evita vazamento de escopo e otimiza o registro e carregamento de funcionalidades. Isso permite que cada parte da sua aplicação seja um módulo isolado e reutilizável, promovendo maior organização, performance e testabilidade.

Outro pilar fundamental é a **validação de dados integrada com JSON Schema**. O Fastify utiliza JSON Schema para validar automaticamente as requisições (headers, params, query, body) e as respostas da API. Isso não apenas garante a integridade dos dados, prevenindo erros comuns e ataques maliciosos, mas também serve como uma documentação viva para seus endpoints, facilitando o desenvolvimento frontend e a integração com outras equipes. A validação é realizada de forma nativa e altamente otimizada, contribuindo para a velocidade geral do framework.

```
1  const path = require('path')
2  const f = require('fastify')({logger: false})
3
4  f.register(require('fastify-static'), {
5    root: path.join(__dirname, 'public'),
6    prefix: '/public/',
7  })
8
9  // In this example, when you get localhost:3000, ou have the time
10 f.get('/', (request, reply) => {
11   reply.header('Content-Type', 'application/json')
12   reply.send({hello: new Date()})
13 })
14 f.get('/about', (request, reply) => {
15   reply.sendFile('about.html')
16 })
17
18
19 const start = async () => {
20   try {
21     await f.listen(3000)
22   } catch (err) {
23     f.log.error(err)
24     process.exit(1)
25   }
26 }
27 start().then(r => r)
```

fastify



Por que Fastify Importa?



Performance Excepcional

Um dos frameworks HTTP mais rápidos do ecossistema JavaScript, oferecendo throughput superior



Baixo Consumo

Projetado para lidar com grande volume de requisições mantendo baixo uso de recursos



Sistema de Plugins

Permite que funcionalidades sejam encapsuladas e reutilizadas de forma modular



Validação Automática

Schemas JSON garantem segurança e confiabilidade dos dados da aplicação



Escalabilidade

Ideal para microserviços e aplicações distribuídas de grande porte

Casos de Uso do Fastify



APIs REST

Para aplicativos web e mobile que necessitam de endpoints rápidos e confiáveis



Microserviços Corporativos

Arquiteturas distribuídas que exigem comunicação eficiente entre serviços



Gateways de API

Camadas de roteamento e agregação para múltiplos backends



Sistemas Distribuídos

Aplicações complexas com componentes independentes comunicando-se via HTTP



Backends de Alto Volume

Sistemas que precisam processar milhares de requisições por segundo

Ferramentas e Ambiente de Desenvolvimento

Neste projeto, utilizaremos ferramentas amplamente aceitas no mercado e de fácil acesso. A escolha adequada do ambiente de desenvolvimento é fundamental para garantir produtividade e boas práticas desde o início do projeto.



Node.js

Ambiente de execução JavaScript utilizado para rodar o servidor Fastify. Versão recomendada: Node 18+ para garantir compatibilidade com as features mais recentes e suporte LTS (Long Term Support).



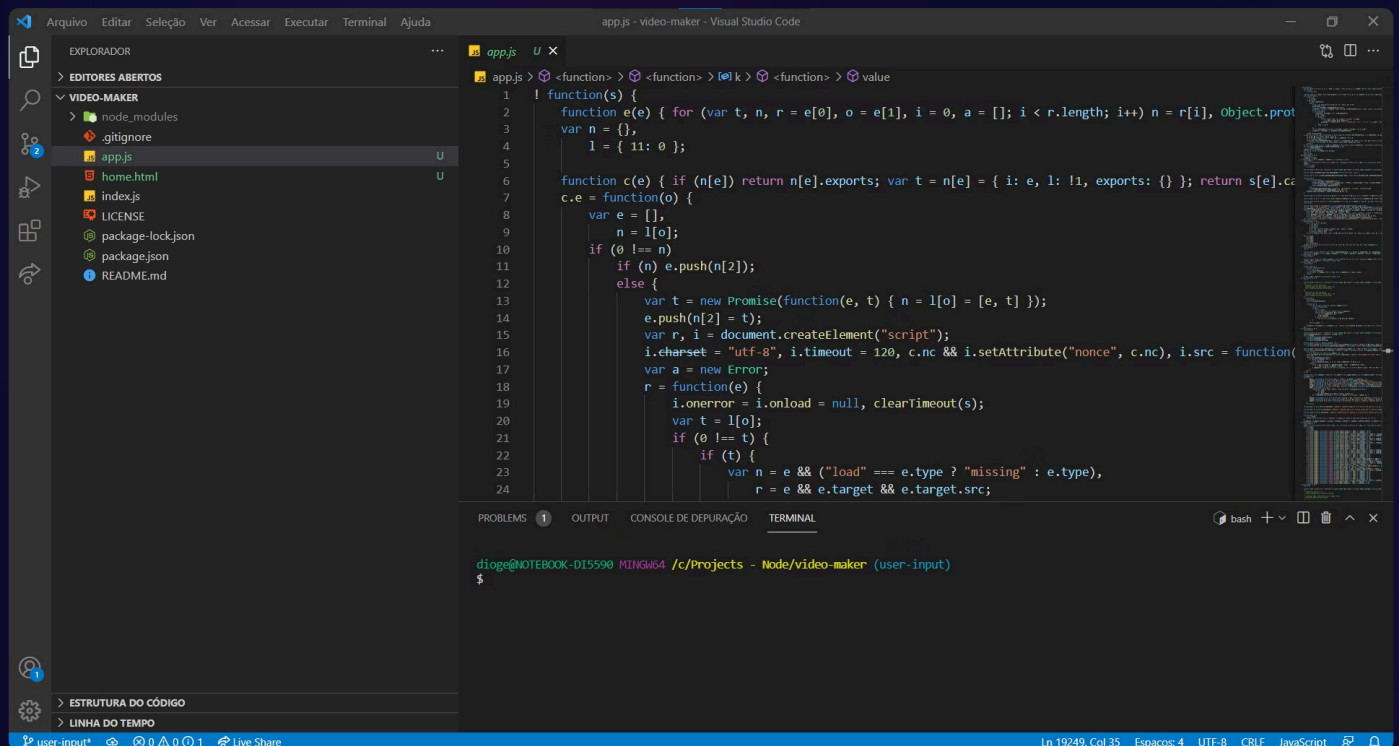
Gerenciador de Pacotes

NPM (padrão do Node) ou PNPM (mais rápido e eficiente). O NPM já vem instalado com o Node.js, mas o PNPM oferece melhor performance e economia de espaço em disco.



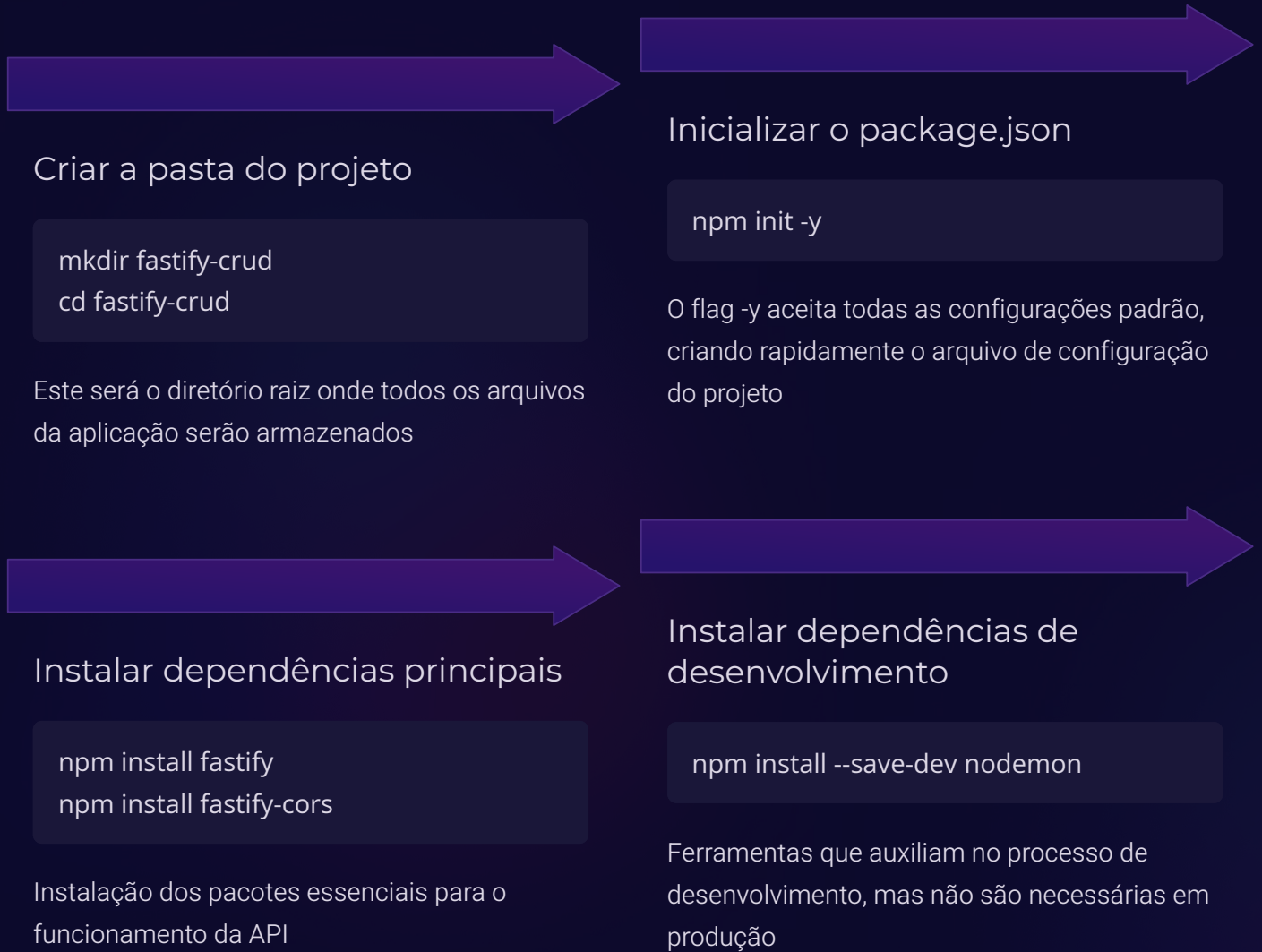
Visual Studio Code

IDE recomendada com extensões essenciais: REST Client para testar APIs, ESLint para garantir boas práticas de código, e Prettier para formatação automática e padronizada.



Setup Inicial do Projeto

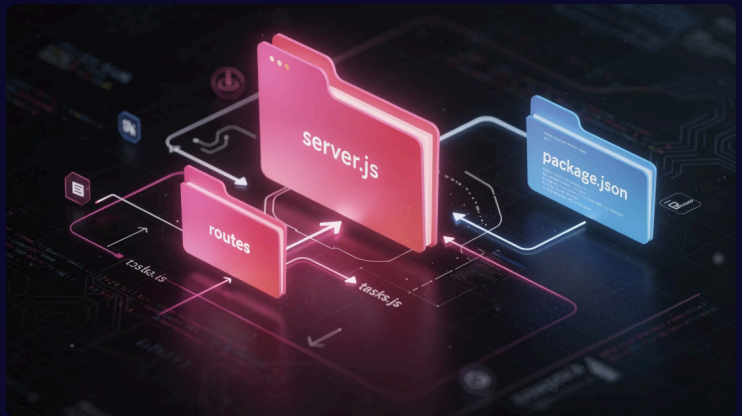
Aqui começa o desenvolvimento real da aplicação. Seguiremos um processo passo a passo para garantir que todos os componentes sejam configurados corretamente desde o início. A organização adequada na fase inicial evita problemas futuros e facilita a manutenção do código.



Estrutura de Pastas da Aplicação

A organização clara e modular da sua aplicação é crucial para o sucesso a longo prazo do projeto. Esta estrutura inicial serve como um alicerce robusto, facilitando a navegação, a colaboração em equipe e a manutenção do código. Ao seguir um padrão desde o início, garantimos que novos desenvolvedores possam entender rapidamente a arquitetura e contribuir de forma eficiente.

A separação lógica de componentes em pastas distintas, como rotas, controladores, modelos e utilitários, é uma prática recomendada (e.g., arquitetura MVC ou camadas) que melhora significativamente a escalabilidade do projeto. Essa modularidade permite que diferentes partes da aplicação sejam desenvolvidas, testadas e mantidas independentemente, minimizando dependências e propagação de erros..



- **server.js:** O ponto de entrada principal da sua aplicação. Este arquivo é responsável por inicializar a instância do Fastify, carregar configurações básicas e iniciar o servidor, atuando como o "bootstrapper".
- **src/:** Contém todo o código-fonte da aplicação. É uma boa prática agrupar a lógica principal aqui para separá-la de arquivos de configuração, testes e ferramentas de build.

Criaremos uma API de tarefas (To-Do) completa com todas as operações básicas de um sistema CRUD. Esta API servirá como exemplo prático de como implementar endpoints RESTful utilizando o Fastify. Para simplificar o desenvolvimento inicial, utilizaremos uma lista em memória para armazenar os dados, eliminando a necessidade de configurar um banco de dados neste momento.



GET /tasks

Listar todas as tarefas cadastradas no sistema



POST /tasks

Criar uma nova tarefa com título e status



PUT /tasks/:id

Atualizar informações de uma tarefa específica



DELETE /tasks/:id

Remover permanentemente uma tarefa do sistema

Configurando o Servidor Principal

O arquivo `server.js` é o coração da aplicação, responsável por inicializar o Fastify, registrar plugins e começar a escutar requisições. Aqui configuramos o logger para monitoramento, habilitamos CORS para permitir requisições cross-origin, e registramos nossas rotas de forma modular.

```
const fastify = require("fastify")({ logger: true });
const cors = require("fastify-cors");

fastify.register(cors);
fastify.register(require('./routes/tasks'));

const start = async () => {
  try {
    await fastify.listen({ port: 3000 });
    console.log("Servidor rodando em http://localhost:3000");
  } catch (error) {
    fastify.log.error(error);
    process.exit(1);
  }
};

start();
```

Pontos Importantes

- Logger habilitado para debugging
- CORS configurado para desenvolvimento
- Tratamento de erros com try-catch
- Uso de async/await para código limpo

Porta de Escuta

Por padrão, o servidor escuta na porta 3000. Em produção, esta porta será fornecida pela plataforma de hosting através da variável de ambiente `process.env.PORT`.

Implementando as Rotas

No arquivo `routes/tasks.js`, implementamos todas as rotas da nossa API. Começamos definindo um array em memória para armazenar as tarefas e exportamos uma função assíncrona que registra todas as rotas no Fastify.

```
let tasks = [
  { id: 1, title: "Estudar Fastify", done: false },
];

module.exports = async function (fastify, opts) {

  // GET - Listar todas as tarefas
  fastify.get("/tasks", async () => {
    return tasks;
  });

  // POST - Criar nova tarefa
  fastify.post("/tasks", async (request, reply) => {
    const { title } = request.body;
    const newTask = {
      id: tasks.length + 1,
      title,
      done: false,
    };
    tasks.push(newTask);
    return newTask;
  });
};
```

1

Listar Tarefas

Retorna o array completo de tarefas armazenadas em memória

2

Criar Tarefa

Extraí o título do body, gera um ID único e adiciona ao array

```
// PUT - Atualizar tarefa existente
fastify.put("/tasks/:id", async (request, reply) => {
  const id = parseInt(request.params.id);
  const { title, done } = request.body;

  const task = tasks.find((t) => t.id === id);
  if (!task) {
    reply.code(404).send({ error: "Task not found" });
    return;
  }
  task.title = title ?? task.title;
  task.done = done ?? task.done;
  return task;
});

// DELETE - Remover tarefa
fastify.delete("/tasks/:id", async (request, reply) => {
  const id = parseInt(request.params.id);
  tasks = tasks.filter((t) => t.id !== id);
  return { message: "Tarefa removida" };
});
};
```

Atualização Parcial

Utiliza o operador nullish coalescing para atualizar apenas os campos fornecidos, mantendo os valores anteriores.

Validação de Existência

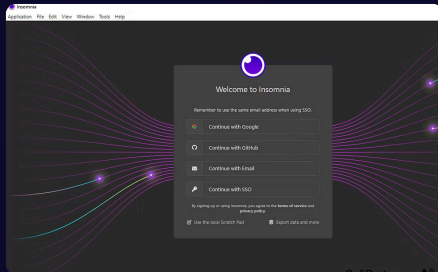
Antes de atualizar, verifica se a tarefa existe e retorna erro 404 caso não encontrada, seguindo boas práticas HTTP

Remoção por Filtro

Usa o método filter para criar um novo array sem a tarefa removida, garantindo imutabilidade dos dados

Testando a API

Existem várias ferramentas profissionais que podem ser usadas para testar sua API de forma eficiente. A escolha da ferramenta depende das preferências pessoais e do contexto do projeto, mas todas oferecem funcionalidades similares para realizar requisições HTTP e validar respostas.



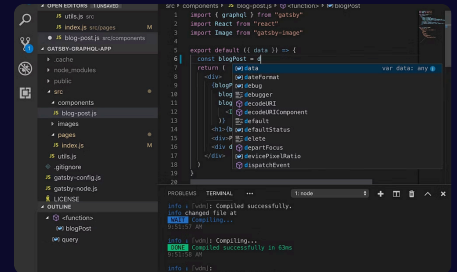
Insomnia

Cliente REST elegante e intuitivo,
ideal para designers e
desenvolvedores que valorizam
interface limpa



Postman

Plataforma completa para
desenvolvimento de APIs com
recursos avançados de
automação e colaboração



REST Client (VS Code)

Extensão leve que permite testar
APIs diretamente no editor,
mantendo requisições
versionadas no Git

Exemplos de Requisições HTTP

Abaixo estão exemplos práticos de como testar cada endpoint da API. Estes exemplos podem ser executados em qualquer cliente HTTP ou copiados para arquivos .http no VS Code usando a extensão REST Client.

Criar Nova Tarefa

```
POST
http://localhost:3000/tasks
Content-Type:
application/json

{
  "title": "Fazer trabalho
de ADS"
}
```

Cria uma nova tarefa com status done=false por padrão

Atualizar Tarefa

```
PUT
http://localhost:3000/tasks/1
Content-Type:
application/json

{
  "done": true
}
```

Marca a tarefa como concluída mantendo o título original

Deletar Tarefa

```
DELETE
http://localhost:3000/tasks/1
```

Remove permanentemente a tarefa do sistema



Dica importante: Ao testar a API, observe os códigos de status HTTP retornados. Status 200/201 indicam sucesso, 404 indica recurso não encontrado, e 500 indica erro no servidor. Sempre valide também o formato do JSON retornado para garantir que a estrutura de dados está correta.

Versionamento com Git e GitHub

O uso de versionamento é essencial em qualquer projeto profissional de software. Git permite rastrear mudanças no código, colaborar com outros desenvolvedores, e manter um histórico completo do desenvolvimento. GitHub adiciona recursos de hospedagem, colaboração visual e integração com ferramentas de CI/CD.



Inicializar Repositório Local

```
git init
git add .
git commit -m "Primeiro commit - estrutura inicial"
```

Cria um repositório Git local e realiza o primeiro commit com todos os arquivos



Conectar com GitHub

```
git branch -M main
git remote add origin https://github.com/usuario/fastify-crud.git
```

Renomeia a branch padrão para main e conecta com repositório remoto



Enviar para Nuvem

```
git push -u origin main
```

Envia todos os commits locais para o GitHub pela primeira vez

Boas Práticas de Versionamento

Estratégias de Commit

- **Commits pequenos e frequentes** facilitam revisão e rollback
- **Mensagens claras** descrevem o que foi feito (ex: "feat: criar rota POST /tasks")
- **Separar branches** por funcionalidades evita conflitos
- **Pull requests** permitem code review antes de merge

Benefícios do Git

Controle de Histórico

Rastreamento completo de todas as mudanças realizadas no projeto

Trabalho em Equipe

Múltiplos desenvolvedores podem trabalhar simultaneamente

Segurança

Backup automático do código na nuvem

Reversão Rápida

Voltar para versões anteriores em caso de problemas

Deploy Automatizado

Integração com plataformas de CI/CD

Deploy na Plataforma Render

A Render é uma plataforma moderna, gratuita e extremamente simples para hospedar APIs Node.js. Ela oferece deploy automático a partir do GitHub, SSL gratuito, e uma interface intuitiva que facilita o gerenciamento de aplicações em produção. A integração nativa com Git permite que cada push para a branch principal dispare um novo deploy automaticamente.

1. Preparar Repositório

Antes de iniciar o deploy, certifique-se de que seu repositório no GitHub está pronto:

- Verifique se o arquivo `package.json` contém um script `start` configurado corretamente (ex: `"start": "node server.js"`).
- Confirme que o arquivo `.gitignore` inclui `node_modules` para evitar o upload de arquivos desnecessários.
- Assegure-se de que todas as dependências do projeto estão listadas em `package.json`.
- Faça um teste final localmente executando `npm start` para garantir que a aplicação funciona como esperado.

3. Novo Web Service

Agora você irá criar o serviço que hospedará sua API:

- No dashboard da Render, clique no botão "New +" e selecione "Web Service".
- Escolha o repositório correto da sua lista de repositórios conectados ao GitHub.
- Selecione a branch que será monitorada para deploys (geralmente `main` ou `master`).
- Defina um nome único para o seu serviço (ex: `minha-api-node` ou seguindo um padrão como `[nome-do-projeto]-api`).

1

2

2. Criar Conta Render

O próximo passo é configurar sua conta Render:

- Acesse render.com e clique em "Get Started" ou "Sign Up".
- Escolha a opção de autenticação via GitHub para uma integração mais fluida.
- Conceda as permissões necessárias para que a Render possa acessar seus repositórios.
- Selecione quais repositórios específicos você deseja permitir que a Render acesse, para manter o controle sobre suas aplicações.

3

4

4. Configurar Build e Runtime

É crucial configurar os comandos e o ambiente para sua aplicação:

- **Build Command:** Defina como `npm install` para instalar todas as dependências do projeto.
- **Start Command:** Especifique o comando de inicialização da sua

Configurações Importantes para Deploy

Para garantir que a aplicação funcione corretamente na Render, algumas configurações específicas são necessárias. A mais importante é a gestão dinâmica de portas, pois plataformas cloud fornecem a porta através de variáveis de ambiente.

Configuração de Porta Dinâmica

A Render e outras plataformas cloud fornecem automaticamente uma porta através da variável de ambiente `PORT`. Seu código deve detectar e usar essa porta ao invés de uma porta fixa. Caso a variável não exista (desenvolvimento local), usa-se 3000 como fallback.

```
await fastify.listen({
  port: process.env.PORT || 3000,
  host: '0.0.0.0'
});
```

- ❏ O host '0.0.0.0' é necessário para permitir conexões externas em ambientes containerizados.

Comandos da Render

Build Command

```
npm install
```

Instala todas as dependências

Start Command

```
node server.js
```

Inicia o servidor em produção

Conclusão

Este projeto permitiu à equipe compreender o ciclo completo de construção de uma aplicação web moderna, iniciando na configuração do ambiente, passando pelo desenvolvimento de rotas, organização do código, testes e boas práticas de versionamento, até o deploy final em uma plataforma de nuvem. A jornada de aprendizado envolveu não apenas aspectos técnicos, mas também o desenvolvimento de habilidades de organização, documentação e trabalho colaborativo.

Principais Aprendizados

- Criação de API REST do zero usando Fastify
- Estruturação modular de rotas e organização de código
- Importância da validação e segurança em APIs
- Técnicas de teste de endpoints corretamente
- Versionamento eficiente de projetos com Git
- Deploy de aplicações em nuvem gratuitamente
- Gestão de configurações entre ambientes

Desafios Enfrentados

- Compreensão do fluxo completo de requisições HTTP
- Organização de código modular em Node.js
- Configuração correta do ambiente de deploy
- Manutenção do controle de versões entre equipe
- Debugging de erros em ambiente de produção



Banco de Dados

Conectar API a MongoDB ou PostgreSQL



Autenticação

Implementar JWT para segurança



Testes Automatizados

Criar suite de testes unitários



Swagger

Documentação automática da API



Microserviços

Evoluir arquitetura distribuída